

```
In [ ]: import json
import os
import pickle
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import FeatureUnion, Pipeline
from sklearn.metrics import classification_report

def load_training_data():
    texts, labels = [], []
    if os.path.exists('train.jsonl'):
        with open('train.jsonl', 'r', encoding='utf-8') as f:
            for line in f:
                item = json.loads(line)
                lbl = 1 if item['label'] > 0.5 else 0
                texts.append(item['text'])
                labels.append(lbl)

    if os.path.exists('train_human_remaining.json'):
        with open('train_human_remaining.json', 'r', encoding='utf-8') as f:
            new_human_data = json.load(f)
            for item in new_human_data:
                texts.append(item['text'])
                labels.append(0)

    return np.array(texts), np.array(labels)
```

```
In [ ]: def build_pipeline():
    features = FeatureUnion([
        ('word', TfidfVectorizer(
            ngram_range=(1, 2),
            max_features=5000,
            sublinear_tf=True,
            dtype=np.float32
        )),
        ('char', TfidfVectorizer(
            analyzer='char',
            ngram_range=(3, 5),
            max_features=12000,
            sublinear_tf=True,
            dtype=np.float32
        ))
    ])

    clf = LogisticRegression(
        C=1.0,
        class_weight='balanced',
        solver='lbfgs',
        max_iter=1000,
        random_state=42,
        n_jobs=1
    )
```

```

return Pipeline([
    ('features', features),
    ('classifier', clf)
])

```

```

In [ ]: def train_model(model_path):
        texts, labels = load_training_data()
        clf = build_pipeline()

        print(f"Training on {len(texts)} samples (all data) ...")
        clf.fit(texts, labels)

        train_acc = clf.score(texts, labels)
        print(f"Training accuracy: {train_acc:.4f}")

        with open(model_path, "wb") as f:
            pickle.dump(clf, f)
        return clf

```

```

In [ ]: def run_benchmark(model_path):
        if not os.path.exists(model_path):
            print("Model file not found!")
            return

        with open(model_path, "rb") as f:
            clf = pickle.load(f)

        test_files = [
            ('eval_human_200.json',      'Human (Label 0)'),
            ('eval_ai_normal.json',      'AI Normal (Label 1)'),
            ('eval_ai_obf.json',         'AI Obfuscated (Label 1)') # fixed typo
        ]

        y_true_all, y_pred_all = [], []
        print(f"\n{'Category':<25} | {'Accuracy':<10}")
        print("-" * 40)

        for file_path, label_name in test_files:
            if not os.path.exists(file_path):
                print(f"{label_name:<25} | FILE NOT FOUND")
                continue

            with open(file_path, 'r', encoding='utf-8') as f:
                data = json.load(f)

            y_true, y_pred = [], []
            for item in data:
                actual = 1 if item['label'] > 0.5 else 0
                prob_ai = clf.predict_proba([item['text']])[0][1]
                prediction = 1 if prob_ai >= 0.5 else 0
                y_true.append(actual)
                y_pred.append(prediction)
                y_true_all.append(actual)
                y_pred_all.append(prediction)

```

```

    acc = np.mean(np.array(y_pred) == np.array(y_true))
    print(f"{label_name:<25} | {acc:.2%}")

    print("-" * 40)
    overall_acc = np.mean(np.array(y_pred_all) == np.array(y_true_all))
    print(f"OVERALL ACCURACY: {overall_acc:.2%}")

    print("\nDetailed Metrics:")
    print(classification_report(y_true_all, y_pred_all, target_names=['Human', 'AI']

if __name__ == "__main__":
    MODEL_FILE = "model.pkl"
    train_model(MODEL_FILE)
    run_benchmark(MODEL_FILE)

```

Training on 27942 samples (all data) ...

c:\Users\anjel\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\linear_model_logistic.py:1184: FutureWarning: 'n_jobs' has no effect since 1.8 and will be removed in 1.10. You provided 'n_jobs=1', please leave it unspecified.
 warnings.warn(msg, category=FutureWarning)

Training accuracy: 0.9941

Category	Accuracy
Human (Label 0)	100.00%
AI Normal (Label 1)	81.00%
AI Obfuscated (Label 1)	48.00%
OVERALL ACCURACY: 76.33%	

Detailed Metrics:

	precision	recall	f1-score	support
Human	0.58	1.00	0.74	200
AI	1.00	0.65	0.78	400
accuracy			0.76	600
macro avg	0.79	0.82	0.76	600
weighted avg	0.86	0.76	0.77	600